Unit III: Search Structures

AVL Trees

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

What is an AVL Tree?

An AVL Tree (named after Adelson-Velsky and Landis) is a self-balancing Binary Search Tree (BST) where the heights of the two child subtrees of any node differ by at most one.

Key Rules & Properties

- **Balance Factor (BF):** For any node N, BF(N) = Height(LeftSubtree) − Height(RightSubtree).
- **AVL Property:** The balance factor of EVERY node must be in {-1, 0, 1}.
- **Height Bound:** The height of an AVL tree with n nodes is strictly bounded by 1.44 $\log_2(n)$, making it highly balanced.
- **Time Complexity:** Search, Insert, and Delete operations all take O(log n) worst-case time.

The Need for Balancing

- **Standard BST Problem:** If keys are inserted in sorted order (e.g. 1, 2, 3, 4), a normal BST degenerates into a linear linked list with O(n) search time.
- **Active Balancing:** AVL trees monitor the balance factor during insertions and deletions.
- **Self-Correction:** If any node's BF becomes +2 or -2, the tree performs localized rotations to restore the AVL balance property immediately.

Single Rotations (LL & RR Cases)

Single rotations are used when the insertion occurs in the 'outer' subtrees (Left-Left or Right-Right):

Right Rotation (LL Case)

- **Trigger:** Node y is inserted into the left subtree of the left child x of root z (Left-of-Left violation). z has BF = +2.
- **Operation:** Pivot around x. Pull z down to become the right child of x. Move x's right child to become z's left child.
- **Result:** x becomes the new root of this subtree, with y on its left and z on its right. Height is balanced.
- **Code Logic:** z.left = x.right; x.right = z;

Left Rotation (RR Case)

- **Trigger:** Node y is inserted into the right subtree of the right child x of root z (Right-of-Right violation). z has BF = -2.
- **Operation:** Pivot around x. Pull z down to become the left child of x. Move x's left child to become z's right child.
- **Result:** x becomes the new root of this subtree, with z on its left and y on its right. Balance restored.
- **Code Logic:** z.right = x.left; x.left = z;

Double Rotations (LR & RL Cases)

Double rotations are required when the insertion occurs in the 'inner' subtrees (Left-Right or Right-Left):

Left-Right Rotation (LR Case)

• **Trigger:** Node is inserted into the right subtree of the left child of root z (Left-Right violation). z has BF = +2.
• **Two Steps to Resolve:**

1. **Left Rotation on Left Child:** Align the imbalance into a straight line (turns LR case into LL case).
2. **Right Rotation on Root z:** Restores the actual height balance of the tree.

• **Final state:** The deepest node is promoted to become the root of this subtree.

Right-Left Rotation (RL Case)

• **Trigger:** Node is inserted into the left subtree of the right child of root z (Right-Left violation). z has BF = -2.
• **Two Steps to Resolve:**

1. **Right Rotation on Right Child:** Align the imbalance into a straight line (turns RL case into RR case).
2. **Left Rotation on Root z:** Restores the actual height balance of the tree.

• **Final state:** The deepest node is promoted to become the root of this subtree.

AVL Insertion & Deletion Complexity

How AVL trees maintain balance during dynamic insertions and deletions:

Insertion Process & Rebalancing

1. Standard BST Insert: Insert the node recursively at a leaf slot. O(log n) search steps.
2. Backtrack and Update: Retrace the path upwards towards the root, updating balance factors.
3. Detect Imbalance: Identify the first node where BF becomes +2 or -2.
4. Perform Rotations: Execute LL, RR, LR, or RL rotation as appropriate. At most ONE rotation is needed to restore balance to the entire tree after an insertion.

Deletion Process & Rebalancing

1. Standard BST Delete: Remove the node. If it has two children, swap with its inorder predecessor or successor, then remove.
2. Retrace and Rebalance: Retrace upwards, updating balance factors.
3. Multiple Rotations: Unlike insertion, a single rotation after a deletion may balance a subtree but reduce its height, causing imbalances further up. We may need to perform up to O(log n) rotations up to the root.

Applications of AVL Trees

AVL trees are highly efficient for lookup-heavy, dynamic key-value storage:

🔍 High-Performance Databases

Used in database indexing where read operations outnumber write operations. AVL trees are more strictly balanced than Red-Black trees, providing faster search lookups (at the cost of slightly slower insertions).

💻 Language Symbol Lookups

Used in language runtimes and memory tables to search for variable names, functions, and object keys where quick access is crucial.

🌐 Network Routing Tables

Router tables use AVL structures to index subnet IPs and prefixes, ensuring rapid packet forwarding routing paths.

🎮 Real-time Systems

Real-time simulation and gaming engines use AVL trees for spatial indexing and quick nearest-neighbor queries where worst-case lookup bounds are strict.

THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management